Université Grenoble Alpes

Compte rendu projet Tents
INF402

Roques Anna
Kërçiku Yuki

Avril 2026

1 Introduction

Pour ce projet, nous avons décidé de nous intéresser au **puzzle Tents**.
Ce jeu se présente sous la forme d'une grille contenant des arbres et dont le but est d'y associer des tentes en respectant différentes contraintes.

Nous chercherons alors à modéliser ce puzzle en logique propositionnelle afin de représenter les contraintes du jeu. Cette modélisation sera ensuite transformée en **forme normale conjonctive**, ce qui permettra d'utiliser par la suite un *solveur SAT* qui sera capable de résoudre le puzzle.

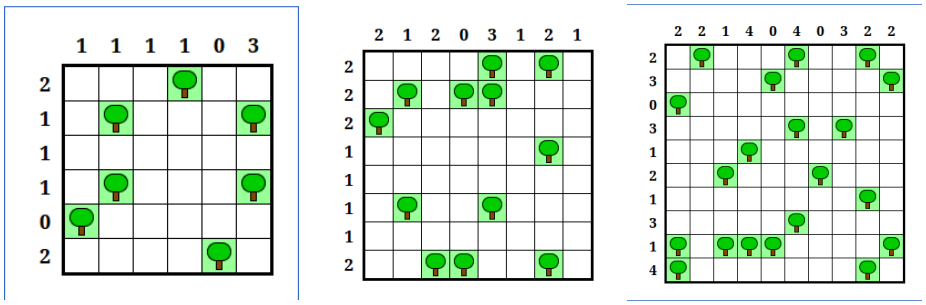
L'objectif de ce compte rendu est de présenter l'ensemble du travail réalisé ,de l'analyse du problème et des choix de modélisation jusqu'à l'implémentation ainsi que l'analyse des résultats obtenus et les perspectives d'amélioration.

2 Présentation du problème

Le puzzle se joue sur une grille $H \times W$ contenant des arbres déjà placés.
L'objectif est alors de placer des tentes sur d'autres cases tout en respectant différentes règles :

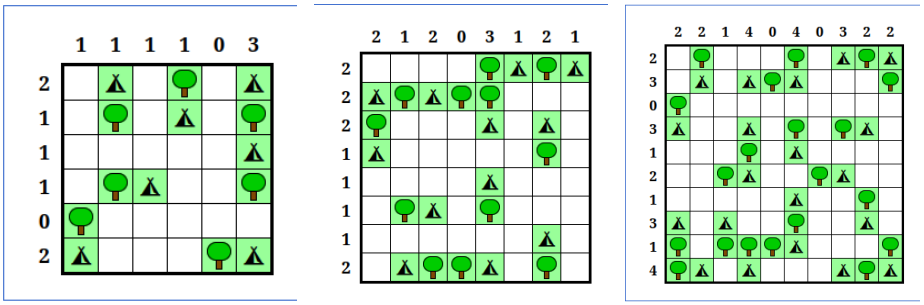
1. une case arbre ne peut pas contenir de tente ;
2. chaque arbre est associé à exactement une tente voisine (haut, bas, gauche, droite) ;
3. une tente est associée à un seul arbre ;
4. deux tentes ne peuvent pas être voisines, même en diagonale ;
5. chaque ligne contient un nombre imposé de tentes ;
6. chaque colonne contient un nombre imposé de tentes.

Exemples Nous présentons ci-dessous trois exemples de grilles initiales ainsi que leurs grilles résolues.



Grille initiale 1 Grille initiale 2 Grille initiale 3

Figure 1: Trois exemples de grilles initiales (source : *puzzle-tents.com*).



Grille résolue 1 Grille résolue 2 Grille résolue 3

Figure 2: Les solutions correspondantes.

3 Formalisation et réduction en forme normale conjonctive (CNF)

Considérons une grille de taille $H \times W$. Chaque case est définie par un couple (i, j) avec :

$$1 \leq i \leq H \qquad 1 \leq j \leq W$$

Certaines cases contiennent des arbres. L'ensemble des positions des arbres est noté $Trees$.

Nous introduisons également :

- R_i le nombre de tentes attendu sur la ligne i;
- C_j le nombre de tentes attendu sur la colonne j.

Et afin de gérer les contraintes de voisinage, sont introduit les ensembles :

- $N_4(i, j)$: voisins orthogonaux d'une case;
- $N_8(i, j)$: voisins incluant aussi les diagonales.

3.1 Variables propositionnelles

Pour chaque case (i, j), considérons la variable propositionnelle :

$$T_{i,j}$$

Qui signifie : $\mathrm{T}_{i,j}$: 'Il y a une tente sur la case (i,j)'
Cette variable est vraie si une tente est placée sur cette case.

Afin de représenter correctement la correspondance entre les arbres et les tentes, est introduite une seconde variable :
Pour chaque arbre t et chaque case vide c voisine,

$$A_{t,c}$$

Qui signifie que l'arbre t est associé à une tente placée sur la case c.

3.2 Contraintes logiques du puzzle

Nous exprimons maintenant les règles du puzzle à l'aide de contraintes logiques sur les variables propositionnelles définies précédemment.

Afin d'utiliser un *solveur SAT*, toutes les contraintes doivent être écrites sous forme normale conjonctive, c'est-à-dire sous la forme :

$$\Gamma = C_1 \wedge C_2 \wedge \cdots \wedge C_n$$

où chaque clause C_i est une disjonction de littéraux. Chaque règle du puzzle est donc traduite en un ensemble de clauses, qui seront ensuite ajoutées à la formule globale Γ.

Une affectation satisfaisant l'ensemble de ces clauses correspond alors à une solution valide du puzzle.

1. Une case contenant un arbre ne peut pas contenir de tente Si une case (i, j) contient un arbre, alors aucune tente ne peut être placée sur cette case. On impose donc :

$$\neg T_{i,j}$$

pour toute case contenant un arbre.

Cette contrainte est déjà une clause en FNC, plus précisément une clause unitaire.

$$(\neg T_{i,j})$$

2.Lien entre une association et la présence d'une tente les variables $A_{t,c}$, où $A_{t,c}$ sont introduites et signifient que l'arbre t est associé à la case c.

Si une association $A_{t,c}$ est vraie, alors la case c doit contenir une tente :

$$A_{t,c} \Rightarrow T_c$$

En FNC, cette implication devient :

$$\neg A_{t,c} \vee T_c$$

On ajoute donc, pour toute association possible entre un arbre t et une case voisine c :

$$(\neg A_{t,c} \vee T_c)$$

3. Une tente doit être associée à un arbre voisin Inversement, si une case c contient une tente, alors cette tente doit être associée à au moins un arbre voisin de c.

On note $N_4(c)$ l'ensemble des arbres voisins de c en voisinage orthogonal, c'est-à-dire en haut, en bas, à gauche ou à droite.

La contrainte logique est :

$$T_c \Rightarrow \bigvee_{t \in N_4(c)} A_{t,c}$$

En FNC, cela donne :

$$\neg T_c \vee \bigvee_{t \in N_4(c)} A_{t,c}$$

On ajoute donc la clause :

$$(\neg T_c \vee A_{t_1,c} \vee A_{t_2,c} \vee \cdots \vee A_{t_m,c})$$

où $t_1, \ldots, t_m$ sont les arbres voisins de la case c.

4. Chaque arbre doit être associé à au moins une tente voisine Chaque arbre doit avoir une tente associée parmi ses cases voisines vides. Pour un arbre t, on note $N_4(t)$ l'ensemble des cases vides voisines de cet arbre.

On impose donc :

$$\bigvee_{c \in N_4(t)} A_{t,c}$$

Cette formule est déjà une clause en FNC :

$$(A_{t,c_1} \vee A_{t,c_2} \vee \cdots \vee A_{t,c_m})$$

où $c_1, \ldots, c_m$ sont les cases voisines vides de l'arbre t.

5. Un arbre ne peut pas être associé à deux tentes différentes Pour deux cases distinctes c et c' voisines du même arbre t, on interdit que les deux associations soient toutes deux vraies :

$$\neg(A_{t,c} \wedge A_{t,c'})$$

En utilisant la loi de De Morgan, on obtient la clause :

$$\neg A_{t,c} \vee \neg A_{t,c'}$$

On ajoute donc, pour chaque paire de cases voisines distinctes c et c' de l'arbre t :

$$(\neg A_{t,c} \vee \neg A_{t,c'})$$

Cette contrainte impose que chaque arbre ait au plus une tente associée.

Avec la contrainte précédente, qui impose au moins une association, on obtient : **exactement une tente associée à chaque arbre**.

6. Une tente ne peut pas être associée à deux arbres différents Pour deux arbres t et t' voisins d'une même case c, on interdit :

$$A_{t,c} \wedge A_{t',c}$$

Ce qui donne en FNC :

$$\neg A_{t,c} \vee \neg A_{t',c}$$

On ajoute donc, pour chaque paire d'arbres t et t' pouvant être associés à la même case c :

$$(\neg A_{t,c} \vee \neg A_{t',c})$$

7. Deux tentes ne peuvent pas se toucher Deux tentes ne peuvent pas être voisines, même en diagonale. On utilise donc le voisinage à 8 directions, noté $N_8(c)$.

Pour deux cases vides voisines c et d, on interdit que les deux contiennent une tente :

$$\neg(T_c \wedge T_d)$$

En FNC, cela donne :

$$\neg T_c \vee \neg T_d$$

On ajoute donc, pour chaque paire de cases voisines c et d en voisinage 8 :

$$(\neg T_c \vee \neg T_d)$$

8. Contraintes sur les lignes et les colonnes Tents impose un nombre exact de tentes par ligne et par colonne.

Pour une ligne i (resp. colonne j), on impose :

$$\sum_j T_{i,j} = R_i \qquad \left(\text{resp. } \sum_i T_{i,j} = C_j\right)$$

Ces contraintes sont traduites en FNC via :

$$\text{exactement } k \equiv (\text{au plus } k) \wedge (\text{au moins } k)$$

Au plus k : on interdit que $k+1$ variables soient vraies simultanément :

$$\bigvee_{x \in S} \neg x \quad \text{pour tout } S,\ S = k+1$$

Au moins k : on interdit que $n-k+1$ variables soient toutes fausses :

$$\bigvee_{x \in S} x \quad \text{pour tout } S,\ S = n-k+1$$

Ainsi, **exactement** k est obtenu en combinant ces deux contraintes.

Dans l'implémentation, cela correspond à :

```
ajout_au_plus_k(...)
ajout_au_moins_k(...)
ajout_exactement_k(...)
```

Les contraintes de lignes et colonnes utilisent donc `ajout_exactement_k`.

4 Formats d'entrée et de sortie

4.1 Format d'entrée du puzzle

Fichier texte (.txt) avec :

- ligne 1 : `H W`;
- ligne 2 : valeurs données par lignes;
- ligne 3 : valeurs données par colonnes;
- puis H lignes de grille ("`T`" pour arbre, "`.`" pour vide)

Code 1: Tests/Grille1_66.txt

```
6 6
2 1 1 1 0 2
1 2 0 2 0 2
T.....
....TT
..T...
......
T..T..
....T.
```

4.2 Format intermédiaire CNF

Le fichier `temp.cnf` est généré au format DIMACS :

- ligne 1 : p cnf ¡nb_vars¿ ¡nb_clauses¿
- puis une ligne par clause, terminée par `0`.

Pour l'instance `Grille1_66`, on obtient par exemple :

Code 2: Extrait de temp.cnf (debut et fin)

```
p cnf 57 371
-1 0
-11 0
-28 0
-35 0
-37 7 0
-38 2 0
...
6 12 24 30 36 0
12 18 24 30 36 0
-6 -12 -18 0
...
-18 -30 -36 0
-24 -30 -36 0
```

Commentaires:

- Les **premières clauses** (`-1 0`, `-11 0`...) sont des clauses unitaires correspondant à la première contrainte.
- Les clauses **au milieu** codent les contraintes principales du puzzle : association arbre–tente, unicité des associations, et interdiction de contact entre les tentes (même en diagonale).
- Les **dernières clauses** codent quant à elles les contraintes de cardinalité (lignes/colonnes).

4.3 Format de sortie solveur

Le fichier `temp.out` contient le resultat du solveur SAT :

- ligne 1 : `SAT` ou `UNSAT`
- puis (si `SAT`) une solution DIMACS

Code 3: Extrait de temp.out

```
SAT
-1 2 -3 -4 -5 6 -7 -8 -9 10 ... -56 57 0
```

Commentaires :

- un entier positif signifie que la variable est vraie ;
- un entier negatif signifie que la variable est fausse ;
- la ligne d'affectation se termine par `0`.

Dans le cas `UNSAT`, aucune affection n'est donnée.

4.4 Sorties utilisateur

- terminal : affichage de la grille solution ;

Code 4: sortie terminal

```
SATISFIABLE
Solution :
T X . . . X
. . . X T T
. X T . . .
. . . X . .
T . . T . .
X . . . T X
```

- fichier `donnees_grilles.js` pour l'affichage graphique ;

Code 5: donnees_grilles.js

```
{
  "indicesLignes": [2, 1, 1, 1, 0, 2],
  "indicesColonnes": [1, 2, 0, 2, 0, 2],
  "grilleInitiale": [
    ["A", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", "A", "A"],
    [".", ".", "A", ".", ".", "."],
    [".", ".", ".", ".", ".", "."],
    ["A", ".", ".", "A", ".", "."],
    [".", ".", ".", ".", "A", "."]
  ],
  "grilleSolution": [
    ["A", "T", ".", ".", ".", "T"],
    [".", ".", ".", "T", "A", "A"],
    [".", "T", "A", ".", ".", "."],
    [".", ".", ".", "T", ".", "."],
    ["A", ".", ".", "A", ".", "."],
    ["T", ".", ".", ".", "A", "T"]
  ]
}
```

- Interface Python :

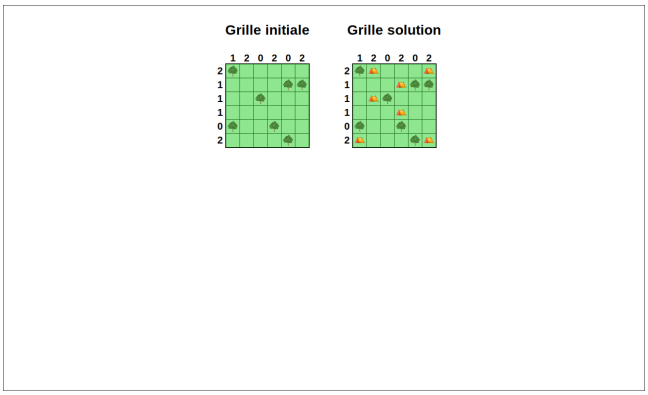


Figure 3: Grille initiale 1

5 Interface homme-machine

Nous avons deux niveaux d'interface :

5.1 Interface en ligne de commande (principale)

Commandes principales :

```
./resolutionTents Tests/Grille1_66.txt
./grille-cnf Tests/Grille1_66.txt temp.cnf
./minisat-grille Tests/Grille1_66.txt temp.out
./cnf-3sat temp.cnf temp_3sat.cnf
```

5.2 Interface graphique (complément)

```
python3 affichage_grilles_tk.py
```

Un script Python (`affichage_grilles_tk.py`) affiche :

- la grille initiale,
- la grille solution.

6 Architecture du code

- `grille.c/.h` : lecture de grille, structure mémoire ;
- `voisinage.c/.h` : voisinages 4 et 8 ;
- `sat.c/.h` : correspondance des variables SAT ;
- `cnf.c/.h` : manipulation clauses/formule ;
- `contraintes.c/.h` : contraintes C1 à C7 ;
- `contraintesCard.c/.h` : contraintes sur lignes/colonnes ;
- `dimacs.c/.h` : écriture CNF, lecture solution ;
- `3sat.c/.h` : transformation en 3-SAT.

7 Implémentation et enchaînement des étapes

7.1 Organisation des exécutables

Le projet présente trois exécutables principaux :

- `grille-cnf` : génère la formule CNF au format DIMACS à partir d'une grille ;

- `minisat-grille` : lit la sortie du solveur et reconstruit la solution sur la grille ;
- `resolutionTents` : exécute automatiquement toute la chaîne de résolution.

Cette séparation permet de vérifier indépendamment la génération de la CNF, la résolution SAT ainsi que l'interprétation de la solution sont sont correctes..

7.2 Exécution étape par étape

1. Génération de la CNF (DIMACS)

```
./grille-cnf Tests/Grille1_66.txt temp.cnf
```

Cet executable lit la grille, construit les variables SAT et les contraintes, puis écrit `temp.cnf`.

2. Résolution SAT puis lecture de la solution

```
minisat temp.cnf temp.out
./minisat-grille Tests/Grille1_66.txt temp.out
```

La première commande permet d'obtenir `temp.out` (`SAT` ou `UNSAT` + affectation).
La seconde interprète cette affectation et affiche la grille solution.

3. Exécution complète automatique

```
./resolutionTents Tests/Grille1_66.txt
```

Cett exectuable enchaîne automatiquement toutes les étapes précédentes : lecture de la grille, génération CNF, appel du solveur, lecture de la solution, affichage final et export des données graphiques.

8 Jeu de tests et cas limites

8.1 Tests standards

Différentes tailles de grilles ont été testées :

- petites : 6x6, 8x8 ;
- moyennes : 10x10, 15x15 ;
- grandes : 20x20, 25x25, 30x30.

8.2 Cas limites SAT/UNSAT

Référencés dans `LISEZMOI.txt`, dont :

- `Grille26_66.txt` : SAT (grille vide, quotas nuls)
- `Grille27_66.txt` : UNSAT (quotas non nuls sans arbres)
- `Grille28_11.txt` : SAT (la plus petite taille valide)
- `Grille29_11.txt` : UNSAT (arbre seul 1x1)
- `Grille30_22.txt` : SAT (petit cas)
- `Grille36_66.txt` : UNSAT (quotas 0 + arbre présent)

8.3 Entrées invalides

- `Grille31_05.txt` : hauteur nulle
- `Grille32_50.txt` : largeur nulle
- `Grille33_malformee_caractere.txt` : caractère invalide
- `Grille34_malformee_ligne_courte.txt` : ligne trop courte
- `Grille35_malformee_indices.txt` : indices incomplets

9 Résultats et observations

9.1 Ce que nous avons observé

- Les instances standards sont correctement résolues.
- Les cas contradictoires sont bien détectés UNSAT.
- Les petits cas (1x1, 2x2, 6x6) sont utiles pour valider les contraintes une par une.
- Le coût de génération CNF augmente exponentiellement avec la taille, en particulier à cause des contraintes de cardinalité.

9.2 Tableau de résultats

Instance	Taille	#Vars	#Clauses	Résultat
Grille1_66	6x6	57	371	SAT
Grille8_88	8x8	104	1076	SAT
Grille11_1010	10x10	166	3543	UNSAT
Grille18_1515	15x15	368	77757	SAT
Grille23_2020	20x20	684	1864970	SAT
Grille24_2525	25x25	1042	35506171	SAT
Grille25_3030	30x30	—	—	—
Grille26_66	6x6	36	218	SAT
Grille27_66	6x6	36	238	UNSAT
Grille28_11	1x1	1	3	SAT
Grille29_11	1x1	1	4	UNSAT
Grille30_22	2x2	6	19	SAT
Grille31_05	0x5	N/A	N/A	Erreur DIMACS
Grille32_50	5x0	N/A	N/A	Erreur DIMACS
Grille33_malformee_caractere	6x6	N/A	N/A	Erreur DIMACS
Grille34_malformee_ligne_courte	6x6	N/A	N/A	Erreur DIMACS
Grille35_malformee_indices	6x6	N/A	N/A	Erreur DIMACS
Grille36_66	6x6	40	221	UNSAT

Table 1: Résultats expérimentaux

Commentaires :
Les lignes SAT/UNSAT montrent que le solveur donne les résultats attendus. Les lignes en `N/A` correspondent à des entrées invalides : aucun DIMACS exploitable n'est crée.

9.3 Interprétation de la limite observée sur les grandes grilles

Un élément important souligné par le tableau obtenu ci-dessus est l'augmentation exponentielle du nombre de clauses générées plus la taille de la grille augmente. Le nombre de clauses passe de 77 757 à 1 864 970, puis à 35 506 171 pour les grilles de tailles (respectivement) 15x15, 20x20 et 25x25. Cette croissance a un impact sur le temps de génération de CNF, et rend la de résolution SAT beaucoup plus coûteuse.

En mesurant le temps de génération CNF avec `grille-cnf` il vient le tableau ci-dessous :

Instance	Entête DIMACS	Nb clauses	Temps génération CNF
`Grille18_1515`	`p cnf 368 77757`	77 757	0.03 s
`Grille23_2020`	`p cnf 684 1864970`	1 864 970	1.04 s
`Grille24_2525`	`p cnf 1042 35506171`	35 506 171	25.74 s

Table 2: Croissance du coût de génération CNF

La grille `Grille25_3030` est très représentative de cette limite. En effet le nombre de clause générées est de l'ordre de $\approx 7 \times 10^8$ avec un temps de génération CNF de ≈ 10 à 15 min. Lorsque l'on execute sur cette grille, les machines arrêtent la génération avec un message d'erreur ou bien e terminal se ferme.

9.4 Origine du coût : encodage des cardinalités

La principale source de ce problème de codage est la méthode utilisée pour les contraintes de cardinalité sur les lignes et colonnes :

$$\text{exactement}_k = (\text{au moins } k) \wedge (\text{au plus } k).$$

Dans notre implémentation actuelle, les contraintes `au_moins_k` et `au_plus_k` sont produites de manière combinatoire. Pour n variables, on obtient un nombre de clauses de l'ordre :

$$\binom{n}{k+1} + \binom{n}{n-k+1}.$$

Ce terme croît donc très vite avec n.

10 Difficultés rencontrées

Les difficultés principales rencontrées pendant le projet ont été de :

- **Faire une correspondance entre la grille et DIMACS.** Le solveur ne manipule en effet que des entiers alors qu'on doit manipuler des positions (i, j) et des associations arbre/case. Il a donc été décidé de définir une numérotation stricte :

  $$T_{i,j} \in [1,\ H \times L], \qquad A_{t,c} \in [H \times L + 1,\ H \times L + N_{\text{assoc}}].$$

  Les variables de tente sont alors générées par `tente_var`, puis les variables d'association par `creer_mapSAT`. Exemple sur `Grille1_66` : $H \times L = 36$, donc $T \in [1, 36]$ et $A \in [37, 57]$.

- **La croissance exponentielle due aux contraintes de cardinalité.** L'encodage de *exactement k* (*au moins k* + *au plus k*) génère de nombreuses combinaisons interdites. Cet encodage est correct, mais pose problème lorsque n est trop grand.

- **Gérer la mémoire en C avec des structures dynamiques volumineuses.** Les clauses et formules CNF sont stockées dans des tableaux dynamiques (`malloc`/`realloc`) qui grandissent progressivement. La difficulté a été de : gérer les échecs d'allocation et bien libérer toutes les clauses générées.

- **Assurer la robustesse pour les grilles avec des entrées invalides.** En testant plusieurs fichiers de test mal formés (dimensions nulles, caractères invalides, lignes incomplètes, indices incohérents) il a été nécéssaire de modifier le code afin d'éviter les incohérences et de retourner les erreurs attendues (`Erreur DIMACS`).

- **Produire un rendu graphique lisible.** Au départ, il avait été décidé de coder cette partie en Java, cependant n'étant pas assez à l'aise avec ce langage et au vu du temps que cela demandait; en décidant finalement de passer à Python cela a simplifié le codage de l'affichage final avec un code plus court et plus claire.

10.1 Choix d'implémentation et pistes d'amélioration

Cette première version pour les contraintes de cardinalité nous a permis de valider rapidement le programme sur des petites grilles mais bien que cette méthode soit correcte elle devient rapidement coûteuse lorsque la taille de la grille est importante générant enormément de clause ce qui ralentit à la fois la génération du fichier CNF et la résolution par MiniSat.

Après analyse du problème et avec l'aide de l'IA nous avons pu voir que d'autres méthodes étaient possibles, plus adaptées et plus efficaces.
Nous avons alrs par la suite essayer une autre méthode, fondée sur un **compteur séquentiel**.
Cette méthode introduit des variables auxiliaires qui permettent de compter progressivement le nombre de variables vraies dans une ligne ou une colonne, sans énumérer toutes les combinaisons.

L'encodage par compteur séquentiel utilisé dans le projet a une complexité en $O(nk)$, où n est le nombre de cases d'une ligne ou d'une colonne et k le nombre de tentes demandé. Pour une grille 30×30, on a au maximum $n = 30$ et $k \leq 30$, donc une contrainte de cardinalité reste au plus de l'ordre de $O(30 \times 30) = O(900)$.

11 Partie optionnelle : CNF vers 3-SAT

Afin de préparer le travail optionnel consistant en la création d'un solveur SAT, il a été demandé de mettre en place une transformation d'une formule CNF en une formule équivalente en 3-SAT.

A donc été implémenter :

```
./cnf-3sat temp.cnf temp_3sat.cnf
```

Cette transformation créée des variables temporaires/auxiliaires permettant de décomposer les clauses de taille quelconque en clauses de taille 3.

Toutefois, par manque de temps, la partie relative à la conception d'un solveur SAT n'a pas pu être faite.

12 Conclusion :

Ce projet nous a permis d'appliquer concrètement les notions de logique propositionnelle vues en cours, notamment la formalisation d'un problème, la traduction des contraintes en forme normale conjonctive (CNF), puis l'utilisation d'un solveur SAT pour obtenir automatiquement une solution et en interpréter le modèle.

À travers le puzzle *Tents*, nous avons construit une chaîne complète de résolution : lecture d'une grille, génération des variables logiques, création de la formule CNF, exécution de MiniSat, puis interprétation et affichage du résultat. Le programme obtenu permet de déterminer si la grille est insatisfiable ou satisfiable et de générée la grille solution.

Les tests réalisés ont montré que l'approche fonctionne correctement sur les petites et moyennes grilles.
Ils ont également montré que la première méthode utilisée pour les contraintes de cardinalité devenait moins efficace lorsque la taille de la grille augmentait, notamment pour les grilles 30×30.
Cela nous a alors conduit à chercher d'autres méthodes plus adaptées comme celle basée sur un **compteur séquentiel**.

13 Références :

- N. Eén and N. Sörensson, MiniSat (solveur SAT utilisé):
  https://github.com/niklasso/minisat
- SAT Competition (moodle INF402):
  https://www.satcompetition.org/
- Format DIMACS CNF (format d'entrée /sortie SAT):
  https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html

14 Commandes utiles

```
make
./resolutionTents Tests/Grille1_66.txt
./grille-cnf Tests/Grille1_66.txt temp.cnf
minisat temp.cnf temp.out
./minisat-grille Tests/Grille1_66.txt temp.out
./cnf-3sat temp.cnf temp_3sat.cnf
python3 affichage_grilles_tk.py
```